

How much detail it takes to fool an adversary? A testbed to evaluate honeytokens effectiveness

Anthi Petridou
Department of Computer Science
and Biomedical Informatics
University of Thessaly
Lamia, Greece
apetridou@uth.gr

Georgios Spathoulas
Department of Information Security
and Communication Technology
NTNU
Gjovik, Norway
georgios.spathoulas@ntnu.no

Abstract—Cybersecurity attacks have consistently increased in number and complexity in recent years. Organizations face a critical challenge to protect their systems from various sophisticated attacks, and one-dimensional protection approaches have been proven to be ineffective. This has led to the introduction of cyber deception techniques as a proactive defense layer utilizing honeytokens. This paper explores the integration of honeytokens into relational database environments through a comparative analysis of different structural and qualitative snapshots of the database. The evaluation of the experiment was conducted in real-world settings with a quantitative assessment of attacker interactions, enabling the identification of specific attack patterns and the physical location of the intruders. The results of this experiment elucidate the structure and usage of honeytokens, as well as how network flow analysis can assist the detection of malicious behavior, resulting in an enhanced practical defensive mechanism.

Index Terms—Cyber deception, Cybersecurity, Databases, Honeytokens, Testbed

I. INTRODUCTION

In recent years, the abrupt advancement and digitalization of modern infrastructures and information systems have caused a significant expansion of the attack surface, making these systems highly attractive targets for malicious actors. Consequently, even though a plethora of traditional defense techniques have been deployed, such as firewalls, advanced techniques to bypass internal security remain a persistent concern in many industries, necessitating a shift from purely reactive to more proactive security paradigms. A crucial aspect of this shift has been network deception [1], which rather than focusing solely on blocking access, its objective is to mislead, confuse, and delay attackers by creating controlled “traps” that attract them. This, in turn, can enable the recording and analysis of their behavior. One of the main approaches for network deception is Honeytokens [2].

Honeytokens constitute a lightweight and flexible form of cyber deception that consists of a digital decoy without a legitimate operational function. Contrary to traditional honeypots [3], which simulate entire systems or services, honeytokens are fabricated or synthetic data, such as database entries, configuration keys, and system files, integrated directly within existing production environments. The importance of honeytokens is highlighted by the fact that they do not

require continuous or active network traffic monitoring and are characterized by low implementation complexity. The purpose of this work is the design, implementation and experimental evaluation of honeytokens embedded in a database environment, combined with the study of their impact on malicious activity detection. The effectiveness of the approach depends on the realism of the fake information added.

The present paper’s contribution is as follows :

- Research on different approaches to create honeytokens information.
- Design and development of honeytokens approaches characterized by different levels of granularity.
- Development of a testbed to assess different honeytokens generation approaches.

The remainder of the paper is structured as follows: Section II discusses the concepts of defensive deception and honeytokens, Section III presents related work, Section IV provides details about the methodology used, Section V presents our results, and finally Section VI concludes the paper.

II. BACKGROUND

A. Deception

Deception is fundamentally a psychological phenomenon of other-induced misperception, where a target is intentionally led to an incorrect hypothesis of reality [4]. Structurally, any act of deception is composed of two simultaneous components: dissimulation, which hides the real, and simulation, which portrays the false. In order to dissimulate, there are only three ways: masking, repackaging, and dazzling [5]. Masking makes the real invisible by either shielding it from sensors or blending it into the background so it is “hidden in plain sight” [4]. Repackaging hides the truth by modifying the appearance of something so that it resembles something else. The third is dazzling, which hides the real by creating confusion or “blurring” a distinctive pattern to reduce the target’s certainty about what they are seeing [4]. In addition, simulation is achieved through three primary methods: mimicking, inverting, and decoying [5]. Mimicking shows the false by having one thing imitate an existing reality. Inventing creates an entirely new reality that is completely false. For example, Honeyd

[6] intercepts connections to unused network addresses and impersonates computers at those addresses. Lastly, decoying diverts the target's attention away from the real setup through distractions.

B. Honeytokens

Honeytokens represent a specialized application of deceptive simulation, defined as digital assets such as database entries, files, or application parameters that possess no legitimate business value and should never be accessed during normal system operations. Structurally, honeytokens function by **Simulation**, where the defender "shows the false" to induce a predictable course of action from an adversary. For example, a web application may include "canary" parameters such as `&Admin=false` within a URL; any modification of this value by a user serves as a deterministic trigger for an alert [2]. This technique works by projecting a false reality, luring attackers towards the non-existent vulnerability.

In order to be more effective, honeytokens must be distributed across numerous architectural layers to establish an in-depth defense. Petrunić [2] organizes these into distinct zones:

- **Zone 2 (Web Application):** The main area for active defense. Using canary parameters, identifies manual or automated scanning procedures and detects malicious activity and potential attacks. It might lead to session termination or sandboxing (monitoring the attacker).
- **Zone 1 & 3 (File System & Database):** Detects whether the application has been breached. Tracks the adversary and collects evidence of attack.
- **Root Level (Robots.txt, as a honeytoken):** Identifies misbehaved "crawlers" and spots various attacks and bot behaviors. It reveals the attacker's IP address.

Although early deception research focused on static traps, modern honeytokens can be classified as either **passive** or **active**. Passive honeytokens are positioned in areas where no legitimate user activity is expected, so any interaction is a confirmed threat. In contrast, active honeytokens are intentionally planted within real data to detect attackers who have successfully gained access to the system. The success of these assets depends on two key properties: believability, where the token must follow statistical patterns of the real data, and it must also be attractive enough to lure an attacker [7].

III. RELATED WORK

The generation and integration of honeytokens has evolved over the past few years as an effective way of protecting a system from unauthorized access. Current research can be categorized into three primary categories:

A. Automated and Data-Level Deception

One of the first completed attempts for automated honeytoken generation is HoneyGen [8]. Honeygen extracts rules from relational databases in order for synthetic data to maintain structural and relational integrity. HoneyGen operates in three phases, Rule Mining, Generation, and Likelihood Rating, in order to produce decoys that are statistically difficult to identify from real records.

B. Machine Learning and Synthetic Data Generation

Recent advances leverage deep learning to enhance the realism of honeytokens. The DpSyn framework [9] utilizes Differentially Private Synthetic Data Generation and deep Auto-Encoders to create relational honeydata that can deceive attackers even when they employ machine learning classifiers for detection. Furthermore, representation learning, such as FastText [10], has been applied to generate realistic honeywords (decoy passwords), significantly increasing the security of hashed password databases.

C. Database Honeytraps and LLM-Powered Deception

Specific research on database deception remains limited. Integrated systems for MySQL [11] utilize both passive honeytokens (unused decoy tables) and active honeytokens (monitored records within production tables) to detect SQL injection and unauthorized queries. Recently, Large Language Models (LLMs) have introduced new capabilities through frameworks such as VellMes [12], which use prompt engineering to simulate realistic network services and generate high-fidelity honeytokens with minimal manual effort.

D. Evaluation of Honeytoken effectiveness from different data stages

Contrary to the related work that has already been presented, the current project focused on how adversaries could detect the implementation of honeytokens through varying placements of the data on the database. More specifically, three different snapshots of the database were created, each of which being more detailed than its predecessor. In addition, a testbed was developed to regulate these snapshots for a more isolated and controlled exposure. In the end, this project would be able to conclude that a more thorough database would imply a more realistic target for malicious activity.

IV. METHODOLOGY

The main research direction of the present work is the relation of the realism of honeytoken artifacts to the efficiency of the deception approach. Database honeytokens can be easily identified based on the coherence of the corresponding records with the rest of the data in the database.

Various tools were used to create realistic honeytoken data for integration into an existing relational database system. An experimental environment that mimics real world infrastructure was developed to ensure a reliable evaluation of honeytoken effectiveness.

Towards that end, the methodology followed is split into the following steps.

- Generation of realistic honeytoken data for a given database
- Deployment of different honeytoken configurations of varying realism
- Development of a testbed that can be used to assess the effectiveness of different honeytoken approaches

A. Data generation

The Northwind sample database served as the base database. It includes 14 tables, each containing different sales data information for a fictional company named "Northwind Traders" [13]. The main tables of the database are:

- Suppliers: Suppliers and vendors of the company.
- Customers: Customers of the company.
- Employees: Details of the employees of the company.
- Products: Product information.
- Shippers: The details of the shippers who transport the products from the company to the end customer.
- Orders and Order_Details: Details of order transactions between company and end customer.

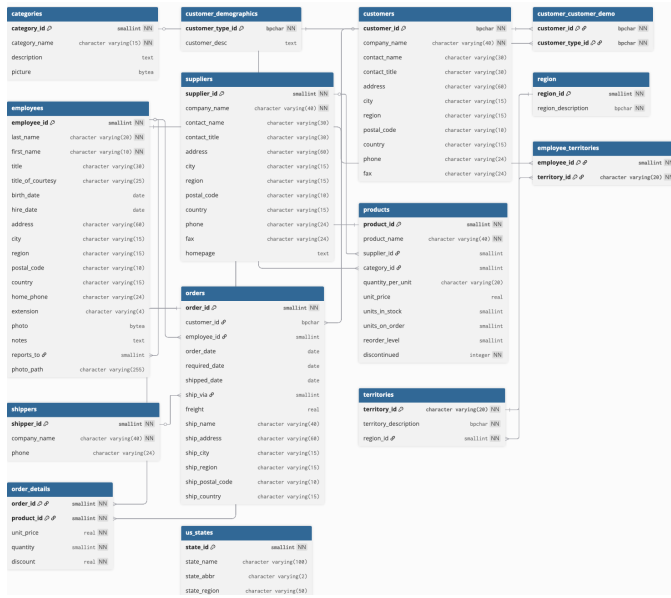


Fig. 1. Entity relationship diagram of the Northwind sample database, including 14 tables.

In addition, we created a new table called "Accounts" as shown in Figure 2, which contained real usernames and passwords of company employees, as well as synthetic data that were used as Honeytokens. The goal of this approach was to increase the "reconnaissance surface" and enhance the believability of the deception. For this to be accomplished, synthetic data were generated through two methods:

- **GAN-based Synthesis:** The CTGAN(Conditional tabular GAN)framework [14] was used to generate new records based on the statistical distributions of the original data.
- **LLM Integration:** Large Language Models (ChatGPT and Gemini) [15] [16] were used to refine the semantic consistency of the generated data, overcoming the limitations of CTGAN regarding logical associations and entity relationships.

accounts	
account_id	int NN
employee_id	smallint NN
username	varchar(50) NN
password_hash	varchar(255) NN

Fig. 2. Table "Accounts".

B. Multi-Snapshot Honeytoken setups

The methodology evaluates the impact of honeytoken placement through three progressive database snapshots:

- 1) **Snapshot 1:** Implementation of an accounts table featuring hashed passwords for existing employees and deceptive "canary" accounts.
- 1) **Snapshot 2:** Expansion of the employee table with synthetic records linked to the accounts table to increase relational depth.
- 1) **Snapshot 3:** Inserting honeytokens in the orders table for complete correlation.

C. Rationale for LLM-Based Data Generation

The decision to utilize ChatGPT to generate the final data set was driven by the limitations observed during initial experiments with the CTGAN library. Although CTGAN is capable of modeling tabular structures, the produced records often lacked full consistency with the database schema and relational constraints.

TABLE I
GENERATED DATA FOR THE FIRST SNAPSHOT USING CTGAN

Employee ID	Username	Password (Hash)
1	nancy.davolio	c2d735ed61274b73ed20...
2	andrew.fuller	920a9b62afa3169f6acb...
3	janet.leverling	356b964e125ff2d6b8ac...
4	margaret.peacock	499bc7df9d8873c1c38e...
5	steven.buchanan	6c94e35ccc352d4e9ef0...
6	michael.suyama	ce8457d59078a699acb7...
7	robert.king	f76cb816b3f74ecf30d3...
8	laura.callahan	7929065522441d4053cb...
9	anne.dodsworth	c75b70bb03291ccdee7c...
10	steven.fuller	f4e99211184a248ac2b1...
11	janet.fuller	7855619d06791afc38b4...
12	nancy.buchanan	62dc22ad2998cc534f7c...
13	anne.buchanan	d1aa36e122cb7e9fdfd...
14	janet.davolio	d54123de468bd42ea00d...

As demonstrated in Table I the generated data (10-14) exhibited a high degree of repetition and low diversity in the generated names. To ensure logical integrity, such as consistency in employee identifiers and demographic attributes, manual corrections were required, making the process more complicated.

In contrast, the use of Large Language Model (LLM) like ChatGPT provided a significantly higher degree of logical coherence and semantic realism (records 10-14, as shown in

Table II). This increased consistency is critical for the effectiveness of honeytokens by ensuring that deceptive assets are statistically and relationally indistinguishable from legitimate production data and the risk of detection through anomaly analysis is minimized. Consequently, the LLM-supported dataset was selected as the optimal choice to evaluate the detection capabilities of the deception system in a realistic environment.

TABLE II
GENERATED DATA FOR THE FIRST SNAPSHOT USING CHATGPT

account_id	employee_id	username	password_hash
1	1	ndavolio	c2d735ed61274b73...
2	2	afuller	920a9b62afa3169f...
3	3	jleverling	356b964e125ff2d6...
4	4	mpeacock	499bc7df9d8873c1...
5	5	sbuchanan	6c94e35ccc352d4e...
16	10	jdoe	ce8457d59078a699...
17	11	asmith	f76cb816b3f74ecf...
18	12	mjones	7929065522441d40...
19	13	kbrown	c75b70bb03291ccd...
20	14	lwilson	f4e99211184a248a...

D. Testbed setup

1) *Concept*: The concept of testbed deployment is to monitor the activity of potential attackers when presented with a given snapshot of the honeytokens. The research hypothesis is that the more realistic the honeytokens solution is, the more probable it is for them to use the fake data they retrieve. In order to achieve this, we designed the following setup:

- A vulnerable and easily accessible database which will hold the generated database snapshots, which include user credentials.
- An authentication web interface on the same host that will probably receive authentication requests with the leaked credentials from the vulnerable database.

By monitoring the activity on the authentication interface, we can draw interesting conclusions about the effectiveness of the honeytokens strategy used and the behavior of potential attackers.

2) *Setup*: The research environment was developed using **Docker containerization** to ensure strict isolation, experimental reproducibility, and controlled exposure of services. The architecture consists of three primary services orchestrated via *docker-compose*:

- **Web Entry Point**: A PHP-based authentication interface (Port 80) that serves as the primary attack vector for the extraction of credentials and web-based reconnaissance.
- **Database Management Systems (DBMS)**: A MySQL instance (Port 3306) that hosts the target data and deceptive assets. Specifically, we loaded the data from the first snapshot (Table II) and created a new user account called “admin”. For this account we set up a weak and insecure password created to simplify any authorized entries.
- **Logging Database**: An independent MySQL instance is used for storing all captured login attempts and audit logs.

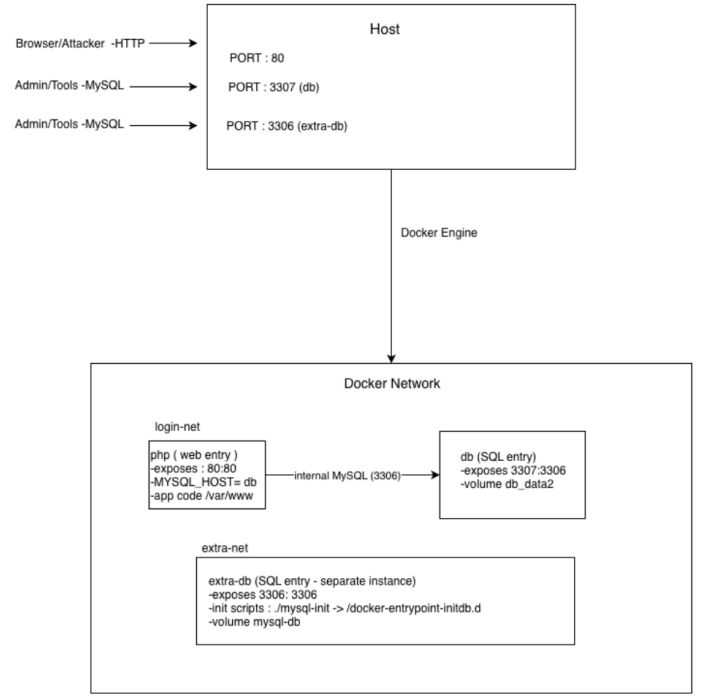


Fig. 3. Architecture of the Containerized Experimental Environment: Web Entry Point (PHP) and SQL Entry Points (DB, Extra-DB) in Isolated Docker Networks (login_net, extra_net) with Port Mapping to the Host.

Listing 1 presents the implementation of the web interface that logs login attempts, while Listing 2 presents the corresponding database table to which the attempts are stored.

```
function log_attempt(mysqli $conn, string $username,
    int $success, ?int $userId = null): void {
    // Prepare the SQL statement for secure
    // insertion
    $sql = "INSERT INTO logins (user_id, username,
        success) VALUES (?, ?, ?)";
    $stmt = $conn->prepare($sql);

    // Bind parameters: 'i' for integer, 's' for
    // string
    $stmt->bind_param("isi", $userId, $username,
        $success);

    // Execute and close the connection to the
    // logging instance
    $stmt->execute();
    $stmt->close();
}
```

Listing 1. PHP function for capturing and persisting login attempts to the isolated logging database.

```
CREATE TABLE IF NOT EXISTS logins (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT,
    username VARCHAR(50) NOT NULL,
    success BOOLEAN NOT NULL,
    login_time DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

Listing 2. SQL schema definition for the logins table used for capturing authentication attempts in the isolated logging database.

Following the principles of Active Defense, the infrastructure was purposefully designed as a vulnerable target to attract malicious activity :

- **Exposed Protocols:** Port 3306 (MySQL) and Port 80 (HTTP) was left open to the internet to allow initial TCP handshakes and automated scanning.
- **Removal of Defense Barriers:** Rate limiting and Intrusion Prevention Systems (IPS) were disabled to allow adversaries to conduct high-volume brute-force attacks and exfiltration attempts without interruption.

V. RESULTS

This section presents the findings derived from an experiment that lasted 46 hours and 37 minutes and during which the deception infrastructure was exposed to live internet traffic.

A. Network Traffic Distribution

During the experimental phase, a total of 170,837 packets were captured. The majority of the network traffic between the two entry points (Port 3306 and Port 80) revealed a significant preference for database services:

- **Port 3306 (MySQL):** Represented 88.4% of total traffic (151,087 packets).
- **Port 80 (HTTP):** Represented 11.7% of total traffic (19,750 packets)

TABLE III
TOTAL PACKET VOLUME PER SERVICE PORT

Service Port	Protocol	Packet Count
80	HTTP	19,750
3306	MySQL	151,087

This gap implies that automated scripts are increasingly bypassing web interfaces to target the database layer, seeking a shorter path to high value data.

B. Attacker Profiling and Specialization

The analysis of 330 unique IP addresses demonstrated a high degree of technical specialization among adversaries.

- **Service Specialization:** 75.1% of unique IPs (248 addresses) interacted exclusively with Port 80 service, while 24.8% (82 addresses) targeted only Port 3306.
- **Minimal Cross-Service Activity:** Only 1.8% of attackers (6 unique IPs) interacted with both Port 80 and Port 3306.

TABLE IV
CONSOLIDATED DISTRIBUTION OF THE 330 UNIQUE IP ADDRESSES BASED ON TARGETED SERVICE AND PERCENTAGE RATIO OF JOINT ACTIVITY.

Activity Category	IP Count	Percentage (%)
Port 80 Exclusive (HTTP)	248	75.1
Port 3306 Exclusive (MySQL)	82	24.8
Joint Activity (Ports 80 & 3306)	6	1.8

C. Analysis of the Attack Patterns

The combination of the login records and the DBMS query logs allowed for the representation of several defined patterns:

- **Brute-Force Attacks:** Many authentication attempts were recorded, utilizing common usernames such as admin, root, and test.
- **Exploitation Techniques:** Captured multiple SQL Injection (e.g., admin' and 'm'='m) and Path traversal attacks (e.g., ../../etc/passwd) at the application layer.
- **Post-Intrusion DBMS Activity:** After successfully connecting to the MySQL instance, a few bots executed environment reconnaissance commands (e.g., SHOW DATABASES) and aggressive administrative actions, including attempts to grant full privileges (GRANT ALL PRIVILEGES) and delete the primary data schema (DROP DATABASE 'data').

VI. DISCUSSION

The study focused only on the first snapshot, as the majority of the detected activity was originated from automated bots and scripts, making the comparison of the three different snapshots incomplete. This indicates a new approach in cyber threats, where automated attacks, enhanced by Artificial Intelligence, will be in control. To conclude, using virtualization technologies, such as Docker containers, has allowed the creation of flexible and reusable system. This architecture can be utilised by other researchers to enhance and extend this defensive mechanism.

This section interprets these findings within the broader context of cyber deception and pinpoints potential modifications for future defensive strategies.

A. Adversary Specialization and Multi-layered Defense

The most remarkable finding of this experiment was that most attackers followed a one dimensional approach. With only 1.8% of unique IP addresses that interact with both HTTP (Port 80) and MySQL (Port 3306) services, it is clear that most modern adversaries, mainly automated bots, leverage highly targeted tools for specific services. Although traditional security techniques, such as the Cyber Kill Chain [13], often foresee a linear attack path that moves from web reconnaissance to database penetration, this experiment accentuated the need for a multi-layered honeypot distribution strategy by revealing that many modern automated threats bypass application-level control to directly target DBMS.

B. Semantic Realism and the Role of Generative AI

An essential component in the success of the deception was the semantic realism of the honeypots. The shift from statistically driven generation (CTGAN) to LLM supported synthesis (ChatGPT) was instrumental in ensuring that deceptive records were in accordance with logical business rules and naming conventions. Nonetheless, the “blind audit” performed by the Gemini LLM uncovered a critical implication: even sophisticated deception can be compromised by the use of overly generic or predictable patterns. While bots were easily

deceived, the LLM's ability to identify inconsistencies in username-employee pairings and naming conventions suggests that as adversaries begin to incorporate AI into their reconnaissance processes, the "bar" for believable deception will continue to rise. This highlights the ongoing need for deception hardening to counter systematic fingerprinting attacks [17] and the implementation of diverse, non-obvious data patterns.

C. Bot Dominance vs. Human Interaction

The findings confirm that the vast majority of current internet wide activity consists of scripts, crawlers, and scanner due to the fact that connection attempts occurred in milliseconds intervals rather than targeted human adversaries. The lack of human interaction means the "psychological" and "behavioral" deceptive qualities of the honeytokens (such as semantic believability) were not truly tested against a thinking opponent.

D. Limitations and Future Directions

The most significant limitation of this study was the primary goal; performing a comparative evaluation of multiple database snapshots with different honeytokens placements was not fully achieved. Because the observed activity was almost entirely automated and focused on initial entry points, the analysis had to be restricted to a single active snapshot. This suggests that the static development of multiple snapshots may require much longer exposure or greater visibility to generate meaningful comparative data. The decision not to use a registered domain name likely limited the system's discoverability. The activity was restricted to adversaries performing mass IP range scanning, which typically uses less sophisticated tools than those used by attackers targeting specific domains. Ultimately, the study established a functional containerized research infrastructure that can serve as a foundation for future work. To fully realize the comparative goals of this research, future experiments should focus on:

- **Extended Duration:** Longer exposure to attract more diverse and targeted traffic.
- **Increased Discoverability:** Utilizing registered domains or publicizing the IP in technical environments to attract human adversaries.

ACKNOWLEDGMENT

This research was partially funded by the European Commission, grant 101120657: European Lighthouse to Manifest Trustworthy and Green AI—ENFIELD

This research was partially funded by the European Commission, grant 101128024: Community for Integrating and Opening Cyber Range Infrastructures that Build an Interoperable Cross-Domain and Cross-Sector Cyber Range Federation—CYBERUNITY.

REFERENCES

- [1] X. Han, N. Kheir, and D. Balzarotti, "Deception techniques in computer security: A research perspective," *ACM Computing Surveys (CSUR)*, vol. 51, no. 4, pp. 1–36, 2018.
- [2] A. R. Petrunić, "Honeytokens as active defense," in *2015 38th International Convention on Information and Communication Technology, Electronics and Microelectronics (MIPRO)*. IEEE, 2015, pp. 1313–1317.
- [3] I. Mokube and M. Adams, "Honeyd: concepts, approaches, and challenges," in *Proceedings of the 45th annual ACM Southeast Conference*, 2007, pp. 321–326.
- [4] B. Whaley, "Toward a general theory of deception," *The Journal of Strategic Studies*, vol. 5, no. 1, pp. 178–192, 1982.
- [5] J. Yuill, D. Denning, and F. Feer, "Using deception to hide things from hackers: Processes, principles, and techniques," *Journal of Information Warfare*, vol. 5, no. 3, pp. 26–40, 2006.
- [6] N. Provos, "Honeyd-a virtual honeypot daemon," in *10th dfin-cert workshop, hamburg, germany*, vol. 2, 2003, p. 4.
- [7] A. Shabtai, M. Bercovitch, L. Rokach, Y. Gal, Y. Elovici, and E. Shmueli, "Behavioral study of users when interacting with active honeytokens," *ACM Transactions on Information and System Security (TISSEC)*, vol. 18, no. 3, pp. 1–21, 2016.
- [8] M. Bercovitch, M. Renford, L. Hasson, A. Shabtai, L. Rokach, and Y. Elovici, "Honeygen: An automated honeytokens generator," in *Proceedings of 2011 IEEE International Conference on Intelligence and Security Informatics*. IEEE, 2011, pp. 131–136.
- [9] N. C. Abay, C. G. Akcora, Y. Zhou, M. Kantarcioglu, and B. Thuraisingham, "Using deep learning to generate relational honeydata," in *Autonomous Cyber Deception: Reasoning, Adaptive Planning, and Evaluation of HoneyThings*. Springer, 2019, pp. 3–19.
- [10] A. Dionysiou, V. Vassiliades, and E. Athanasopoulos, "Honeygen: Generating honeywords using representation learning," in *Proceedings of the 2021 ACM Asia conference on computer and communications security*, 2021, pp. 265–279.
- [11] M. Wegerer and S. Tjoa, "Defeating the database adversary using deception—a mysql database honeypot," in *2016 International Conference on Software Security and Assurance (ICSSA)*. IEEE, 2016, pp. 6–10.
- [12] M. Sladić, V. Valeros, C. Catania, and S. Garcia, "Vellmes: A high-interaction ai-based deception framework," in *2025 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*. IEEE, 2025, pp. 671–679.
- [13] Microsoft, "Northwind sample database," 2024, accessed: 2026-05-09. [Online]. Available: <https://docs.yugabyte.com/stable/develop/sample-data/northwind/>
- [14] J. Rivers, A. Nelson, and L. Williams, "Synthetic data generation with sdv," 2020.
- [15] OpenAI. (2026) Chatgpt (gpt-4o). Accessed: May 10, 2026. [Online]. Available: <https://chatgpt.com>
- [16] Google. (2026) Gemini (gemini 1.5 pro). Accessed: May 10, 2026. [Online]. Available: <https://gemini.google.com>
- [17] S. Srinivasa, J. M. Pedersen, and E. Vasilomanolakis, "Towards systematic honeytokens fingerprinting," in *13th International Conference on Security of Information and Networks*, 2020, pp. 1–5.